ReadMe

aeon_to_rein.py – AEON → RE:IN + JSON

This script converts an AEON model (.aeon) into a RE:IN file (.rein) and an intermediate JSON file (.json) used later by rein_add_experiments.py.

Default: preserves certainty from the AEON file

- interactions with ? → written as optional
- interactions without ? → written as definite (no optional)

Expected folders:

- input: aeon/<MODEL_NAME>.aeon
- output: rein/<MODEL_NAME>.rein and rein/<MODEL_NAME>.json

Run Example:

python aeon_to_rein.py --model-name 023

Force all interactions to be optional:

python aeon_to_rein.py --model-name 023 --all-optional

sbml_to_rein.py – SBML → RE:IN + JSON

Running on SBML is the same as aeon_to_rein.py, just use **sbml_to_rein.py** and place the input under sbml/<MODEL_NAME>.sbml (instead of aeon/<MODEL_NAME>.aeon).

rein_add_experiments.py (INIT → FIN)

Generates a .rein file with **N** synthetic experiments:

- random unique INIT states
- simulate **K** steps
- observe FIN at time **K**

Run Example:
python rein_add_experiments.py --model-type aeon --model-name 023 --k 10 --n 20

rein_add_experiments_with_midSteps.py (INIT → MID → FIN)

Same as above, but observations at **0**, **k-mid**, **k-end**.

Run Example:
python rein_add_experiments_with_midSteps.py --model-type aeon --model-name 023 --k-mid 2 --k-end 4 --n 5

General Flags

- **--model-name <NAME>**
    - **Description: Required.** Specifies the name of the model you want to process (do not include the .aeon or .sbml extension).
    - **Relevant Scripts:** aeon_to_rein.py, sbml_to_rein.py, rein_add_experiments.py
    - **Example:** python aeon_to_rein.py --model-name 023

2. Conversion Flags

- **--all-optional**
    - **Description: Optional.** Overrides the default parsing behavior. Instead of preserving the original certainty, it forces *all* interactions in the model to be written as optional (adding ?).
    - **Relevant Scripts:** aeon_to_rein.py, sbml_to_rein.py
    - **Example:** python sbml_to_rein.py --model-name 023 --all-optional

3. Experiment Generation Flags

- **--N <INT>** *(Note: Adjust this flag name if you used something else in your code)*
    - **Description: Optional.** Defines the number of random, unique initial (INIT) states to generate.
    - **Relevant Scripts:** rein_add_experiments.py
    - **Example:** --N 10
- **--K <INT>** *(Note: Adjust this flag name if you used something else in your code)*
    - **Description: Optional.** Defines the number of steps to simulate for each trajectory before observing the final (FIN) state.
    - **Relevant Scripts:** rein_add_experiments.py
    - **Example:** --K 5

4. Three-Timepoint Experiment Flags

These flags are specific to the rein_add_experiments_with_midSteps.py script, which records data at three points (INIT, MID, FIN).

- **--k-mid <INT>** * **Description: Optional.** Defines the timepoint (step number) for the intermediate (MID) observation. Must be ≥ 0. (Default: 2).
    - **Relevant Scripts:** rein_add_experiments_with_midSteps.py
    - **Example:** --k-mid 3
- **--k-end <INT>** * **Description: Optional.** Defines the timepoint (step number) for the final (FIN) observation. Must be $\ge$ --k-mid. (Default: 4).
    - **Relevant Scripts:** rein_add_experiments_with_midSteps.py
    - **Example:** --k-end 6